UNIVERSITY INSTITUTE of COMPUTING
Asia's Fastest Growing University

NAAC GRADE A+

WorkSheet - 6

Student Name: Arnab Lala **UID: 25MCI10232**
Branch: M.C.A(A.l & M.L) **Section/Group: MAM-1 A**
Semester: SEM II **Date of Performance:24/02/2026**
Subject Name: TECHINCAL SKILLS Subject Code: 25CAP-652_25MAM_KAR-1(A)

AIM: Learn how to create, query, and manage views in SQL to simplify database queries and provide a layer of abstraction for end-users.

S/W Requirement: Oracle Database Express Edition and pgAdmin

OBJECTIVES:

- Data Abstraction: To understand how to hide complex table joins and calculations behind a simple virtual table interface.
- Enhanced Security: To learn how to restrict user access to sensitive columns by providing views instead of direct table access.
- Query Simplification: To master the creation of views that pre-join multiple tables, making reporting easier for non-technical users.
- View Management: To understand the syntax for creating, altering, and dropping views, as well as the naming conventions required for efficient data access.

Practical / Experiment Steps

```
-- STEP 1 — Create Categories Table
CREATE TABLE Categories (
   category_id SERIAL PRIMARY KEY,
   category_name VARCHAR(100)
);
INSERT INTO Categories (category_name) VALUES
('Electronics'),
('Clothing'),
('Groceries'),
('Home Decor');
-- STEP 2 — Create Products Table
CREATE TABLE Products (
   product_id SERIAL PRIMARY KEY,
   product_name VARCHAR(100),
   price NUMERIC(10,2),
   stock_quantity INT,
   category_id INT,
```

```
 FOREIGN KEY (category_id) REFERENCES Categories(category_id)
);
INSERT INTO Products (product_name, price, stock_quantity, category_id) VALUES
('Smartphone', 25000, 15, 1),
('Jeans', 1800, 40, 2),
('Rice Bag 5kg', 600, 100, 3),
('Table Lamp', 1200, 8, 4),
('Bluetooth Headphones', 3500, 5, 1);
-- STEP 3 — Simple Filter View (Low Stock View)
-- Create a view to show low stock products.
CREATE OR REPLACE VIEW low_stock_products AS
SELECT product_id, product_name, stock_quantity
FROM Products
WHERE stock_quantity < 10;
SELECT * FROM low_stock_products;
```

	product_id integer	product_name character varying (100)	stock_quantity integer
1	4	Table Lamp	8
2	5	Bluetooth Headphones	5

```
-- STEP 4 — Join View (Product + Category)
CREATE OR REPLACE VIEW product_category_view AS
SELECT
    p.product_id,
    p.product_name,
    p.price,
    p.stock_quantity,
    c.category_name
FROM Products p
JOIN Categories c
ON p.category_id = c.category_id;
SELECT * FROM product_category_view;
```

	product_id integer	product_name character varying (100)	price numeric (10,2)	stock_quantity integer	category_name character varying (100)
1	1	Smartphone	25000.00	15	Electronics
2	2	Jeans	1800.00	40	Clothing
3	3	Rice Bag 5kg	600.00	100	Groceries
4	4	Table Lamp	1200.00	8	Home Decor
5	5	Bluetooth Headphones	3500.00	5	Electronics

```
-- STEP 5 — Aggregate View (Category Statistics)
CREATE OR REPLACE VIEW category_inventory_summary AS
SELECT
```

```
c.category_name,
    COUNT(p.product_id) AS total_products,
    SUM(p.stock_quantity) AS total_stock,
    AVG(p.price) AS average_price,
    MAX(p.price) AS highest_price,
    MIN(p.price) AS lowest_price
FROM Categories c
LEFT JOIN Products p
ON c.category_id = p.category_id
GROUP BY c.category_name;
SELECT * FROM category_inventory_summary;
```

	category_name character varying (100)	total_products bigint	total_stock bigint	average_price numeric	highest_price numeric	lowest_price numeric
1	Electronics	2	20	14250.0000000000000000	25000.00	3500.00
2	Groceries	1	100	600.0000000000000000	600.00	600.00
3	Clothing	1	40	1800.0000000000000000	1800.00	1800.00
4	Home Decor	1	8	1200.0000000000000000	1200.00	1200.00

```
-- STEP 6 — Create Sales Table
CREATE TABLE Sales (
    sale_id SERIAL PRIMARY KEY,
    product_id INT,
    quantity_sold INT,
    sale_date DATE DEFAULT CURRENT_DATE,
    FOREIGN KEY (product_id) REFERENCES Products(product_id)
);
INSERT INTO Sales (product_id, quantity_sold, sale_date) VALUES
(1, 3, '2026-03-01'),
(2, 5, '2026-03-01'),
(1, 2, '2026-03-02'),
(3, 10, '2026-03-02'),
(5, 4, '2026-03-03'),
(4, 1, '2026-03-03');
-- STEP 7 — Revenue View
CREATE OR REPLACE VIEW sales_revenue_summary AS
SELECT
    p.product_name,
    SUM(s.quantity_sold) AS total_units_sold,
    SUM(s.quantity_sold * p.price) AS total_revenue
FROM Sales s
JOIN Products p
ON s.product_id = p.product_id
```

```
GROUP BY p.product_name;

SELECT * FROM sales_revenue_summary;
```

	product_name character varying (100)	total_units_sold bigint	total_revenue numeric
1	Smartphone	5	125000.00
2	Table Lamp	1	1200.00
3	Jeans	5	9000.00
4	Rice Bag 5kg	10	6000.00
5	Bluetooth Headphones	4	14000.00

```
-- STEP 8 — Top Selling Products View

CREATE OR REPLACE VIEW top_selling_products AS

SELECT

    p.product_name,

    SUM(s.quantity_sold) AS total_units_sold

FROM Sales s

JOIN Products p

ON s.product_id = p.product_id

GROUP BY p.product_name

ORDER BY total_units_sold DESC;

SELECT * FROM top_selling_products;
```

	product_name character varying (100)	total_units_sold bigint
1	Rice Bag 5kg	10
2	Smartphone	5
3	Jeans	5
4	Bluetooth Headphones	4
5	Table Lamp	1

LEARNING OUTCOMES:

- Abstraction Proficiency: Students will be able to create and query views to simplify efficient data access and abstraction.
- Security Implementation: Students will understand how to use views for data masking and providing restricted access to sensitive information.
- Syntactic Accuracy: Students will demonstrate the correct syntax for creating and querying views, ensuring logical clarity in naming conventions.
- Real-world Application: Students will be able to design views for practical domains like Library Management Systems or Payroll Systems to demonstrate functionality.